



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

Основная образовательная программа 09.03.02

«Информационные системы и технологии»

**Отчет по дисциплине «Алгоритмы и структуры обработки данных»
по лабораторной работе №3**

Тема: «Сортировки»

Проверил
к.т.н. доцент

Ковалев И.А.

подпись

Выполнила
студентка группы ИДБ-25-06

Якунина В.С.

подпись

Москва, 2026

Оглавление

Введение.....	3
Тексты программ.....	4
912. Sort an Array.....	4
88. Merge Sorted Array.....	6
977. Squares of a Sorted Array.....	8
56. Merge Intervals.....	9
452. Minimum Number of Arrows to Burst Balloons.....	11
75. Sort Colors.....	13
Вспомогательные функции.....	14
Функция с тестами.....	15
Заключение.....	18

Введение

Целью работы является изучение методов сортировок и использование их на практике. Работа выполнена на C++ и частично на Python.

Тексты программ

912. Sort an Array

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int* arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

int* sort_an_array(int* arr, int n) // heap_sort
{
    // Здесь подойдет пирамидальная сортировка

    for (int i = n / 2 - 1; i > -1; i--) {
        heapify(arr, n, i);
    }

    for (int i = n - 1; i > -1; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }

    return arr;
}
```

Программа сортирует массив целых чисел пирамидальной сортировкой.

Результаты тестирования представлены на рисунках 1, 2.

```
10, 3, 2, 6, 1, 8, 9
```

Входные данные, рисунок 1

```
Sort an Array  
1 2 3 6 8 9 10
```

Выходные данные, рисунок 2

88. Merge Sorted Array

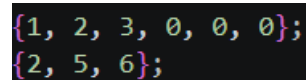
```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort_inplace(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort_inplace(arr, low, pi - 1);
        quick_sort_inplace(arr, pi + 1, high);
    }
}

void merge_sorted_array1(int nums1[], int m, int nums2[], int n) {
    // складываем все в один список O(n + n log n)
    for (int i = 0; i < n; i++) {
        nums1[m + i] = nums2[i];
    }
    // и сортируем его
    quick_sort_inplace(nums1, 0, m + n - 1);
}
```

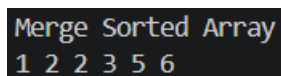
Программа принимает на вход 2 массива целых чисел, все данные добавляет в один из них и сортирует in-place сортировкой.

Результаты тестирования программы показаны на рисунках 3, 4.



```
{1, 2, 3, 0, 0, 0};
{2, 5, 6};
```

Входные данные. Рисунок 3



```
Merge Sorted Array
1 2 2 3 5 6
```

Выходные данные. Рисунок 4

977. Squares of a Sorted Array

```
void square(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * arr[i];
    }
}

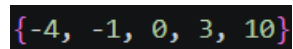
int* square_of_a_sorted_array1(int* arr, int n) {
    // 1 способ: в квадрат и сортируем  $O(n + n \log n)$ 
    square(arr, n);

    // пирамидальная сортировка из первого задания
    sort_an_array(arr, n);

    return arr;
}
```

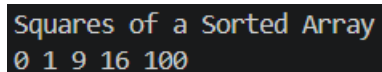
Программа возводит элементы массива в квадрат и сортирует их по возрастанию пирамидальной сортировкой.

Результаты тестирования показаны на рисунках 5, 6.



`{-4, -1, 0, 3, 10}`

Входные данные. Рисунок 5



`Squares of a Sorted Array`
`0 1 9 16 100`

Выходные данные. Рисунок 6

56. Merge Intervals

```
int max(int a, int b) {
    if (a > b) return a;
    return b;
}

void swap_intervals(int a[2], int b[2]) {
    int temp[2] = {a[0], a[1]};
    a[0] = b[0];
    a[1] = b[1];
    b[0] = temp[0];
    b[1] = temp[1];
}

void quick_sort(int a[][2], int lo, int hi) {
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2][1]; // опорный – средний элемент
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i][1] < pivot) i++;
        while (a[j][1] > pivot) j--;
        if (i <= j) {
            swap_intervals(a[i], a[j]);
            i++;
            j--;
        }
    }
    quick_sort(a, lo, j);
    quick_sort(a, i, hi);
}

int** merge_intervals(int arr[][2], int n, int* res_len) {
    quick_sort(arr, 0, n - 1);

    int j = 0;
    int** res = new int*[n];
    for (int i = 0; i < n; i++) {
        res[i] = new int[2];
    }

    res[0][0] = arr[0][0];
    res[0][1] = arr[0][1];
    for (int i = 0; i < n; i++) {
        if (arr[i][0] <= res[j][1]) {
            res[j][1] = max(res[j][1], arr[i][1]);
        } else {
            j++;
            res[j][0] = arr[i][0];
        }
    }
    *res_len = j + 1;
    return res;
}
```



```

        res[j][1] = arr[i][1];
    }
}

*res_len = j + 1;
return res;
}

```

Программа принимает массив интервалов и объединяет их.

Результаты тестирования показаны на рисунках 7, 8.

```

{{1, 3}, {2, 6}, {8, 10}, {15, 18}}

```

Входные данные. Рисунок 7

```

Merge Intervals
{ { 1, 6 }, { 8, 10 }, { 15, 18 } }

```

Выходные данные. Рисунок 8

452. Minimum Number of Arrows to Burst Balloons

```
void qs_rec(int a[][2], int lo, int hi) {
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2][1]; // опорный – средний элемент
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i][1] < pivot) i++;
        while (a[j][1] > pivot) j--;
        if (i <= j) {
            swap_intervals(a[i], a[j]);
            i++;
            j--;
        }
    }
    qs_rec(a, lo, j);
    qs_rec(a, i, hi);
}

int min_num_of_arrows_to_burst_balloons(int balloons[][2], int n) {
    qs_rec(balloons, 0, n - 1);

    int i = 0, res = 0;
    int temp;
    while (i < n) {
        temp = balloons[i][1];
        i++;
        if (balloons[i][0] <= temp && temp <= balloons[i][1]) {
            i++;
            res++;
        }
    }

    return res;
}
```

Программа принимает массив диаметров воздушных шаров, висющих на стене, и вычисляет, за какое минимальное количество стрел возможно лопнуть все шарики.

Результаты тестирования представлены на рисунках 9, 10.

```
{{10, 16}, {2, 8}, {1, 6}, {7, 12}}
```

Входные данные. Рисунок 9

```
Minimum Number of Arrows to Burst Balloons  
2
```

Выходные данные. Рисунок 10

75. Sort Colors

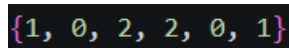
```
int partition_colors(int* arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort_inplace_colors(int* arr, int low, int high) {
    if (low < high) {
        int pi = partition_colors(arr, low, high);
        quick_sort_inplace_colors(arr, low, pi - 1);
        quick_sort_inplace_colors(arr, pi + 1, high);
    }
}

void sort_colors(int* arr, int n) {
    quick_sort_inplace_colors(arr, 0, n - 1);
}
```

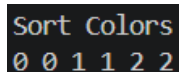
Программа принимает массив цветов, обозначенных «0», «1» и «2», и сортирует оптимизированной быстрой сортировкой (in-place).

Результаты тестирования представлены на рисунках 11, 12.



A black rectangular box containing the text "{1, 0, 2, 2, 0, 1}" in a monospaced font. The numbers are colored: 1 is red, 0 is green, 2 is blue, 2 is blue, 0 is green, and 1 is red.

Входные данные. Рисунок 11



A black rectangular box containing the text "Sort Colors" in a monospaced font. Below it, the numbers "0 0 1 1 2 2" are displayed in a monospaced font, where the first two 0s are green, the two 1s are red, and the two 2s are blue.

Выходные данные. Рисунок 12

Вспомогательные функции

```
void print_array(int* arr, int n) {  
    for (int i = 0; i < n; i++) {  
        std::cout << arr[i] << " ";  
    }  
}
```

Функция выводит в консоль массив в форматированном виде.

```
void clear_matrix(int** arr, int n)  
{  
    for (int i = 0; i < n; i++)  
    {  
        delete[] arr[i];  
    }  
    delete[] arr;  
}
```

Функция чистит динамическую память.

Функция с тестами

```
int main() {
    int n = 0, m = 0;

    // 1
    std::cout << "Sort an Array" << std::endl;
    n = 7;
    int arr1[] = {10, 3, 2, 6, 1, 8, 9};

    print_array(sort_an_array(arr1, n), n);

    std::cout << std::endl;

    // 2
    std::cout << "Merge Sorted Array" << std::endl;
    n = 3;
    m = 3;
    int arr21[] = {1, 2, 3, 0, 0, 0};
    int arr22[] = {2, 5, 6};
    merge_sorted_array1(arr21, m, arr22, n);
    print_array(arr21, m + n);

    std::cout << std::endl;

    // 3
    std::cout << "Squares of a Sorted Array" << std::endl;

    n = 5;
    int arr3[] = {-4, -1, 0, 3, 10};

    print_array(square_of_a_sorted_array1(arr3, n), n);
    // print_array(square_of_a_sorted_array2(arr3, n), n);

    std::cout << std::endl;

    // 4
    std::cout << "Merge Intervals" << std::endl;

    n = 4;
    int res_len;
    int intervals[][2] = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};

    int** result = merge_intervals(intervals, n, &res_len);
    std::cout << '{';
    for (int i = 0; i < res_len; i++) {
        std::cout << " { " << result[i][0] << ", " << result[i][1];
        if (i == res_len - 1) {
            std::cout << " } ";
        }
    }
}
```

```

        } else {
            std::cout << " },";
        }
    }
    std::cout << '}'';
    clear_matrix(result, res_len);

    std::cout << std::endl;

    // 5
    std::cout << "Minimum Number of Arrows to Burst Balloons" <<
std::endl;

    n = 4;
    int balloons[][2] = {{10, 16}, {2, 8}, {1, 6}, {7, 12}};
    std::cout << min_num_of_arrows_to_burst_balloons(balloons, n) <<
std::endl;

    // 6
    std::cout << "Sort Colors" << std::endl;

    n = 6;
    int arr6[] = {1, 0, 2, 2, 0, 1};
    sort_colors(arr6, n);

    print_array(arr6, n);

    std::cout << std::endl;

    return 0;
}

```

Программа тестирует каждую из представленных выше программ и выводит результат в консоль.

Результаты общего тестирования представлены на рисунке 13.

```
Sort an Array
1 2 3 6 8 9 10
Merge Sorted Array
1 2 2 3 5 6
Squares of a Sorted Array
0 1 9 16 100
Merge Intervals
{ { 1, 6 }, { 8, 10 }, { 15, 18 } }
Minimum Number of Arrows to Burst Balloons
2

Sort Colors
0 0 1 1 2 2
```

Рисунок 13

Программа выбора сортировки в зависимости от входных данных

```
def switch_sort(arr, n):
    if n < 50:
        bubble_sort(arr, n)
    elif n < 100:
        selection_sort(arr)
    elif n <= 150:
        insertion_sort(arr)
    elif n <= 500:
        quick_sort(arr, n)
    elif n <= 750:
        quick_sort_inplace(arr, 0, n - 1)
    elif max(arr) - min(arr) < 100:
        counting_sort(arr)
    elif n <= 1000:
        merge_sort(arr, n)
    elif n <= 2000:
        heap_sort(arr, n)
    elif n <= 1000000:
        radix_sort(arr)

# 1
def bubble_sort(arr, n):
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

# 2
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# 3
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

```

4

```

def quick_sort(arr, n):
    if n <= 1:
        return arr

    pivot = arr[n // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

```

5

```

def quick_sort_inplace(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort_inplace(arr, low, pi - 1)
        quick_sort_inplace(arr, pi + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

```

6

```

def merge_sort(arr, n):
    if n <= 1:
        return arr

    mid = n // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:

```

```

        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# 7
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr, n):
    # Построение max-heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Извлечение элементов
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

    return arr

# 8
def counting_sort(arr):
    if not arr:
        return arr

    max_val = max(arr)
    min_val = min(arr)
    range_size = max_val - min_val + 1

    count = [0] * range_size
    output = [0] * len(arr)

    # Подсчёт элементов
    for num in arr:
        count[num - min_val] += 1

```

```

# Кумулятивный подсчёт
for i in range(1, len(count)):
    count[i] += count[i - 1]

# Построение выходного массива
for num in reversed(arr):
    output[count[num - min_val] - 1] = num
    count[num - min_val] -= 1

return output

# 9
def counting_sort_for_radix(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    for i in range(n):
        index = arr[i] // exp
        count[index % 10] += 1

    for i in range(1, 10):
        count[i] += count[i - 1]

    i = n - 1
    while i >= 0:
        index = arr[i] // exp
        output[count[index % 10] - 1] = arr[i]
        count[index % 10] -= 1
        i -= 1

    for i in range(n):
        arr[i] = output[i]

def radix_sort(arr):
    if not arr:
        return arr

    max_val = max(arr)
    exp = 1

    while max_val // exp > 0:
        counting_sort_for_radix(arr, exp)
        exp *= 10

    return arr

def main():
    print("Введите количество элементов:")

```

```

n = int(input())
print("Введите элементы:")
arr = [int(input()) for _ in range(n)]

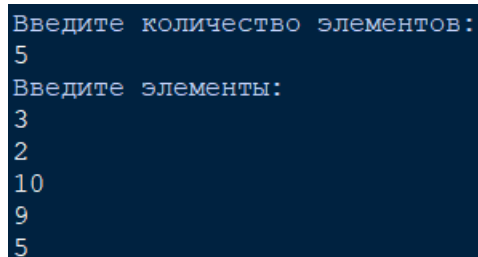
switch_sort(arr, n)
print(arr)

if __name__ == "__main__":
    main()

```

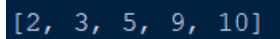
Программа принимает массив целых чисел и его длину, а далее сортирует его по наиболее подходящей сортировке.

Результаты тестирования представлены на рисунках 14, 15.



Введите количество элементов:
5
Введите элементы:
3
2
10
9
5

Входные данные. Рисунок 14



[2, 3, 5, 9, 10]

Выходные данные. Рисунок 15

Заключение

В данной лабораторной работе мы изучили методы сортировки списков и применили их на практике.